COMPUTER ASSIGNMENT-2

AM5630 – Foundations of Computational Fluid Dynamics

BY:

KARTHICK.J (OE23S018)

M.S SCHOLAR,

DEPARTMENT OF OCEAN ENGINEERNIG,

IIT MADRAS

ASSIGNMENT -2

PROBLEM DEFINITION:

Consider a rectangular plate of unit thickness with dimensions 1 x 2 units (2-D steady state diffusion problem, Boundary value problem).

a) Plot the steady state temperature contour of the rectangular plate for the following boundary conditions.

b) Calculate the heat transfer along the bottom boundary of the plate. (Take k = 50 W/mK)

c) Include the tabulated data shown at the end of the document.

GOVERNING EQUATION:

$$\frac{\partial^2 T}{\partial X^2} + \frac{\partial^2 T}{\partial y^2} = 0$$

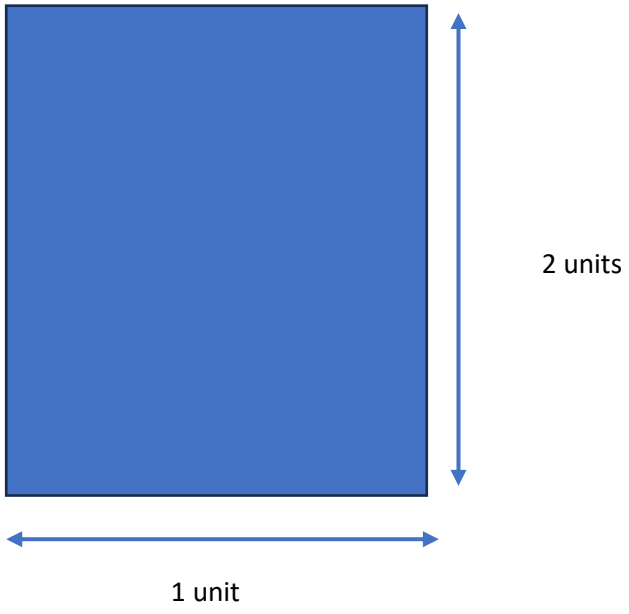
BOUNDARY CONDITION:

Bottom boundary = 100℃

All other boundaries = 30 ℃

DOMAIN SIZE:

Grid size = 21 x 41



CONVERGENCE CRITERIA:

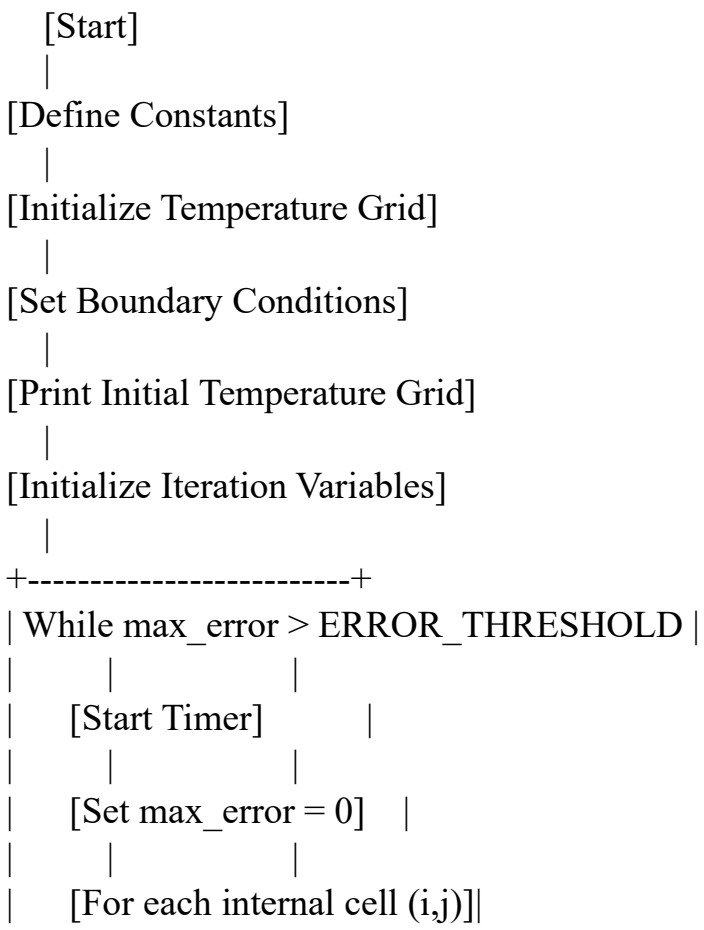
Error = $\sum_{i=2,j=2}^{i=iM-1,j=jN-1} \left| T_{i,j}^{k+1} - T_{i,j}^{k} \right|$

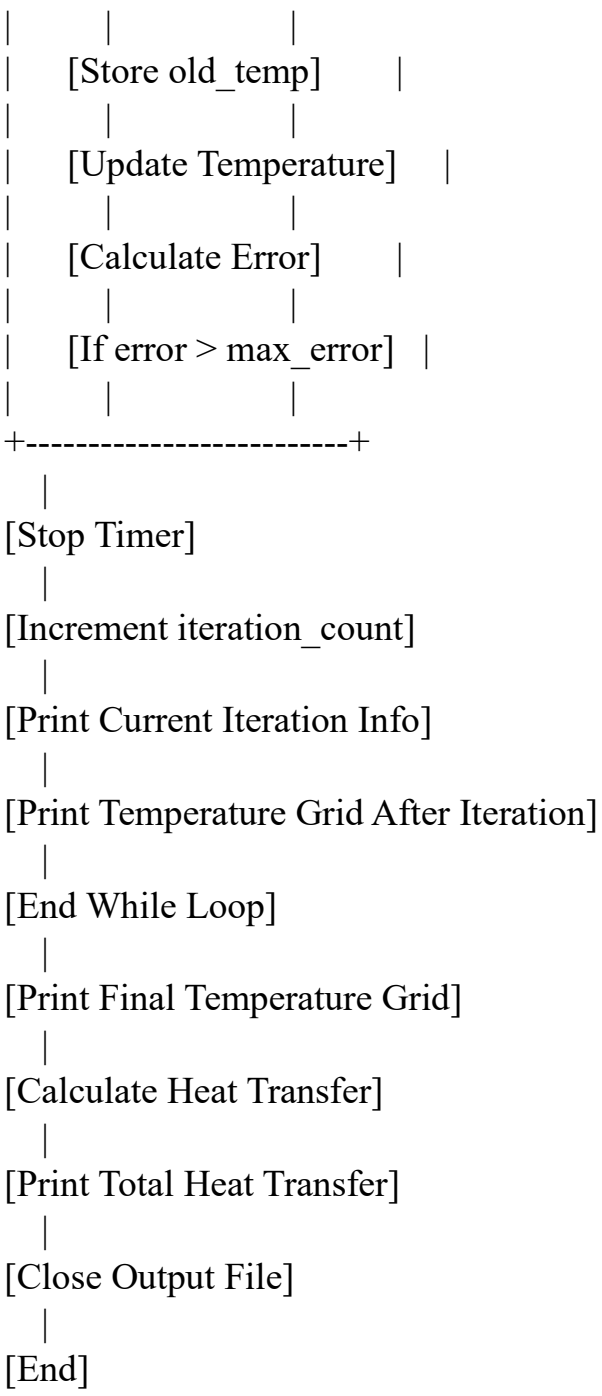
SCHEMES ARE IMPLEMENTED:

- Point Gauss Seidel
- Line Gauss Seidel
- PSOR
- LSOR
- ADI
- ADI with relaxation

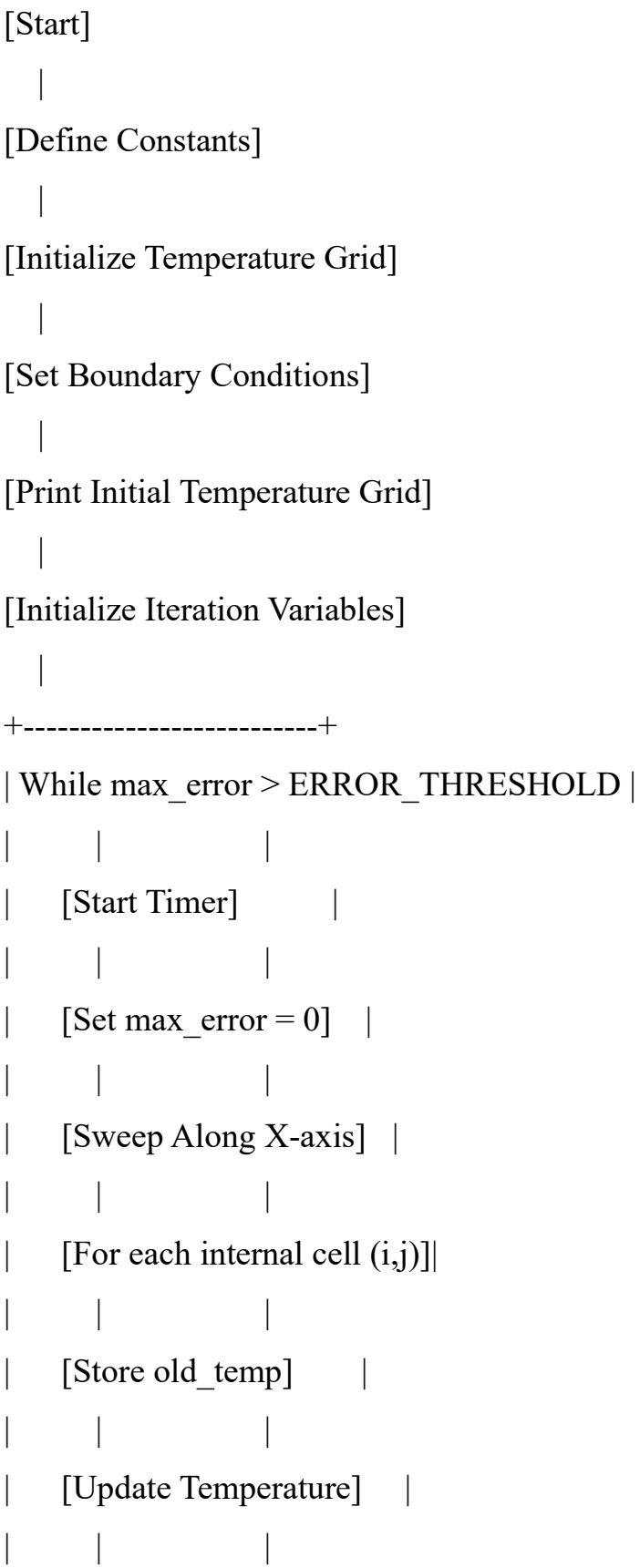
ALGORITHM FOR THE SCHEMES:

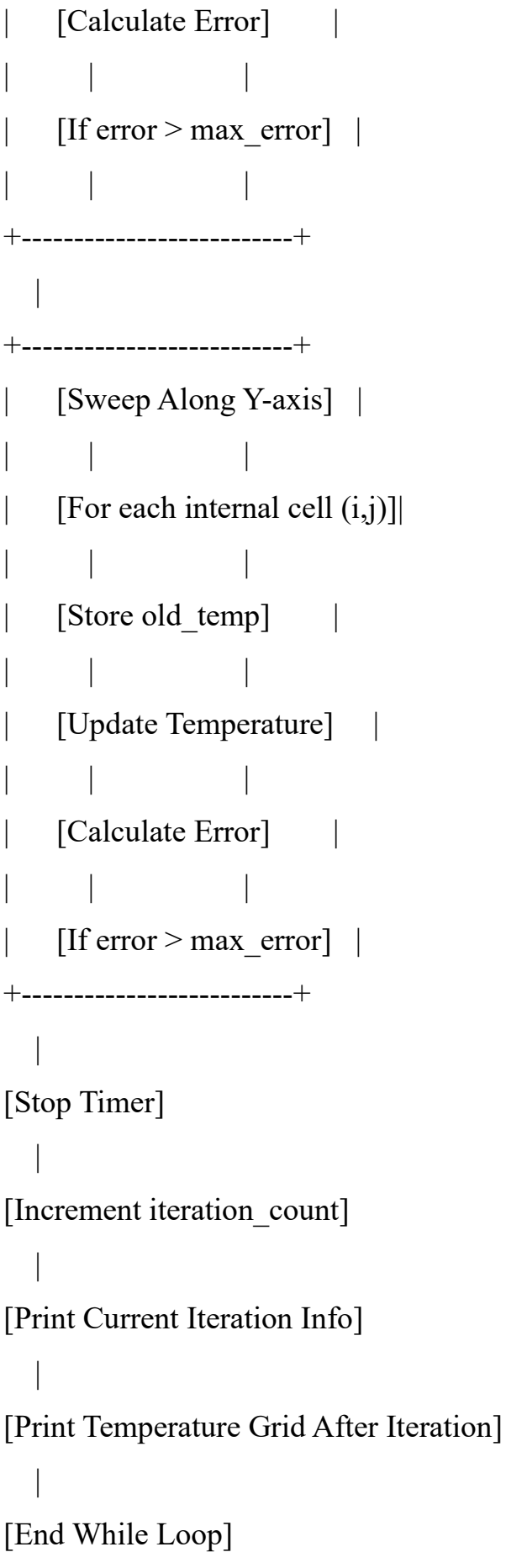
1. Point Gauss seidel Method:




[Store old_temp]
[Update Temperature]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]
[End While Loop]
[Print Final Temperature Grid]
[Calculate Heat Transfer]
[Print Total Heat Transfer]
[Close Output File]
[End]

Line Gauss Siedel method:




[Calculate Error]
[If error > max_error]
[Sweep Along Y-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]
[End While Loop]

|

[Print Final Temperature Grid]

|

[Calculate Heat Transfer]

|

[Print Total Heat Transfer]

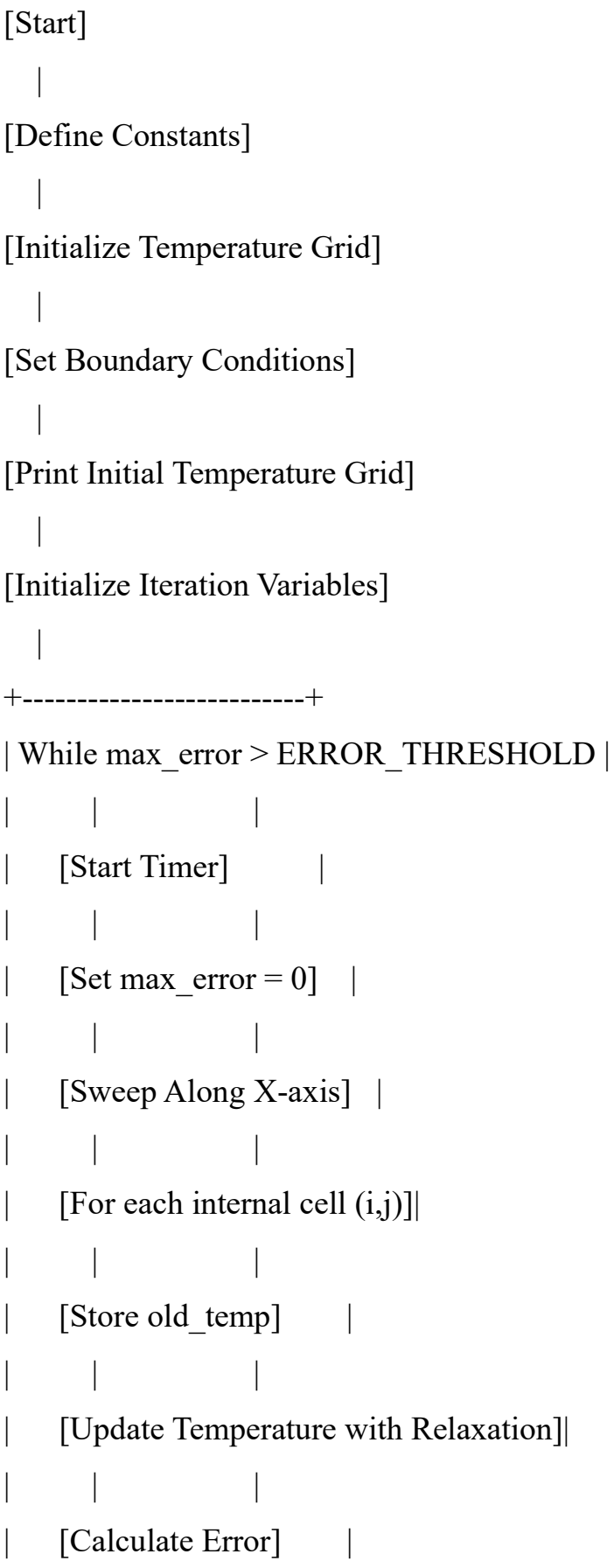
|

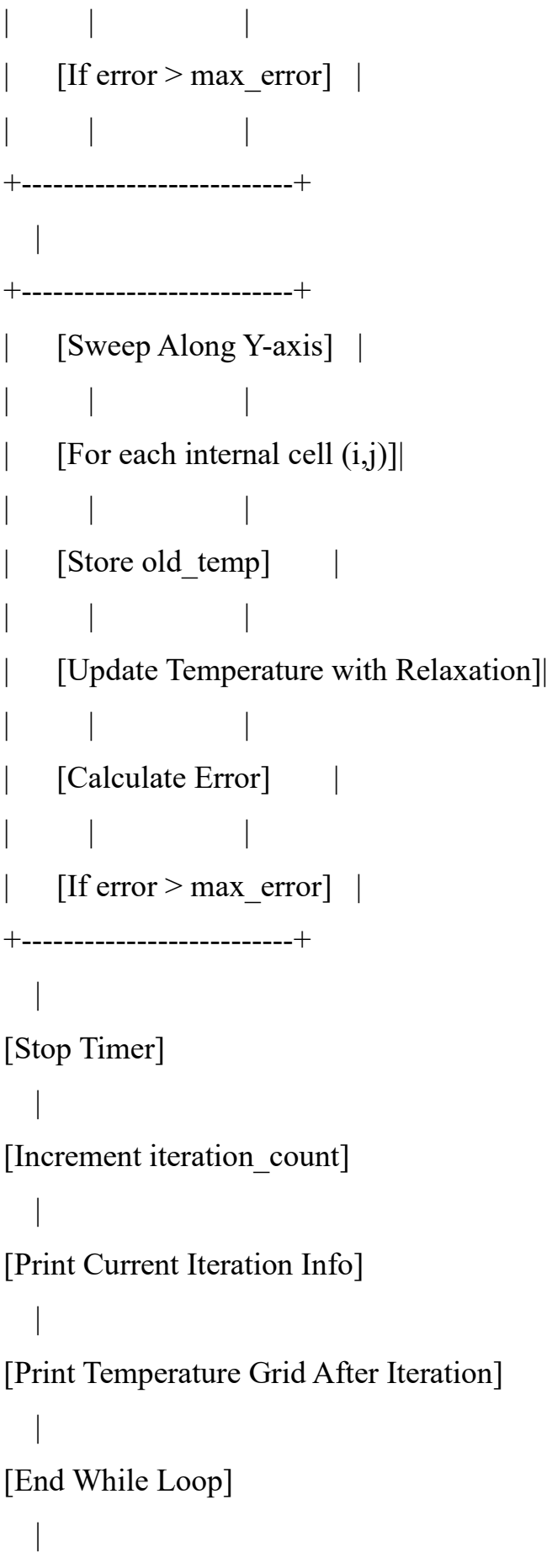
[Close Output File]

|

[End]

PSOR method:


[Start]
[Define Constants]
[Initialize Temperature Grid]
[Set Boundary Conditions]
[Print Initial Temperature Grid]
[Initialize Iteration Variables]
While max_error > ERROR_THRESHOLD
[Start Timer]
[Set max_error = 0]
[Sweep Along X-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]
[Calculate Error]


[If error > max_error]
[Sweep Along Y-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]
[End While Loop]

[Print Final Temperature Grid]

|

[Calculate Heat Transfer]

|

[Print Total Heat Transfer]

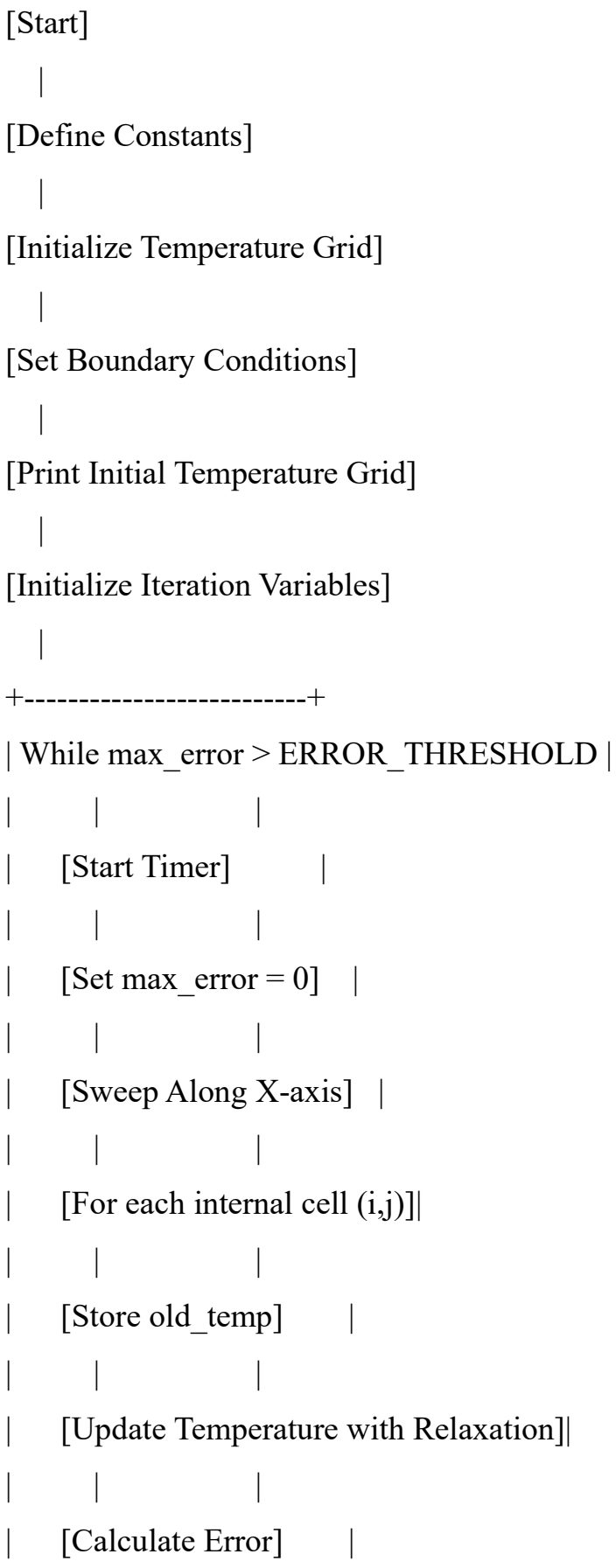
|

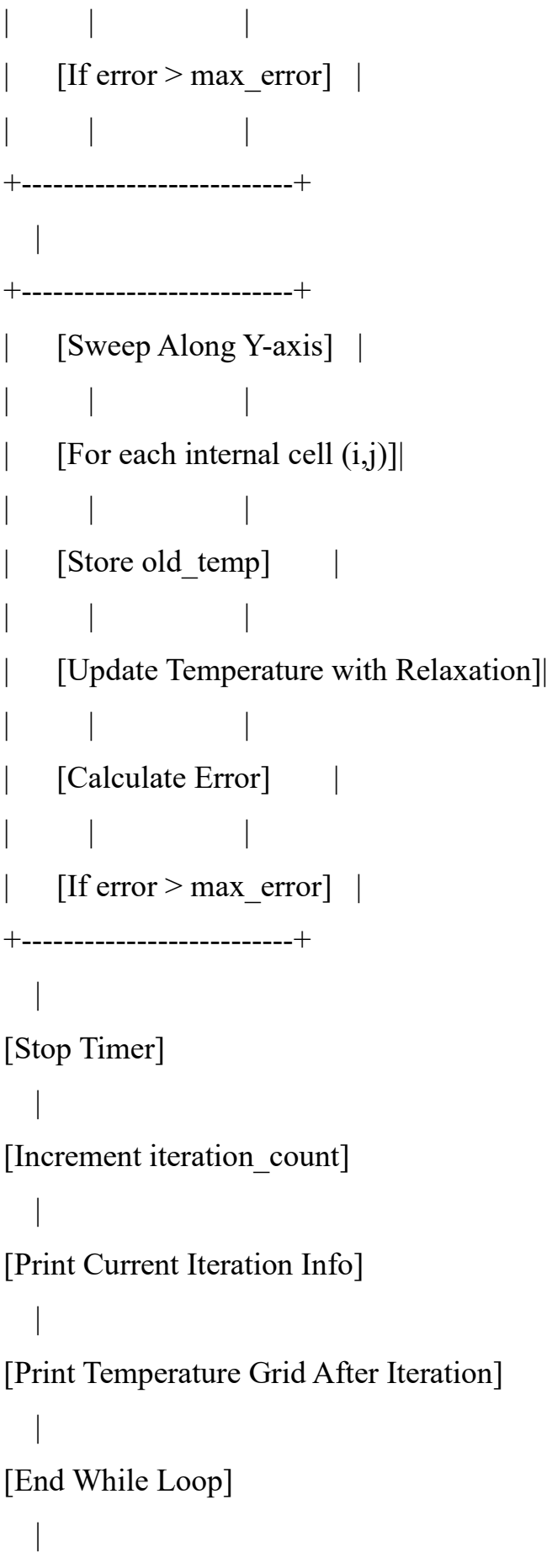
[Close Output File]

|

[End]

LSOR method:


[Start]
[Define Constants]
[Initialize Temperature Grid]
[Set Boundary Conditions]
[Print Initial Temperature Grid]
[Initialize Iteration Variables]
While max_error > ERROR_THRESHOLD
[Start Timer]
[Set max_error = 0]
[Sweep Along X-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]
[Calculate Error]


[If error > max_error]
[Sweep Along Y-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]
[End While Loop]

[Print Final Temperature Grid]

|

[Calculate Heat Transfer]

|

[Print Total Heat Transfer]

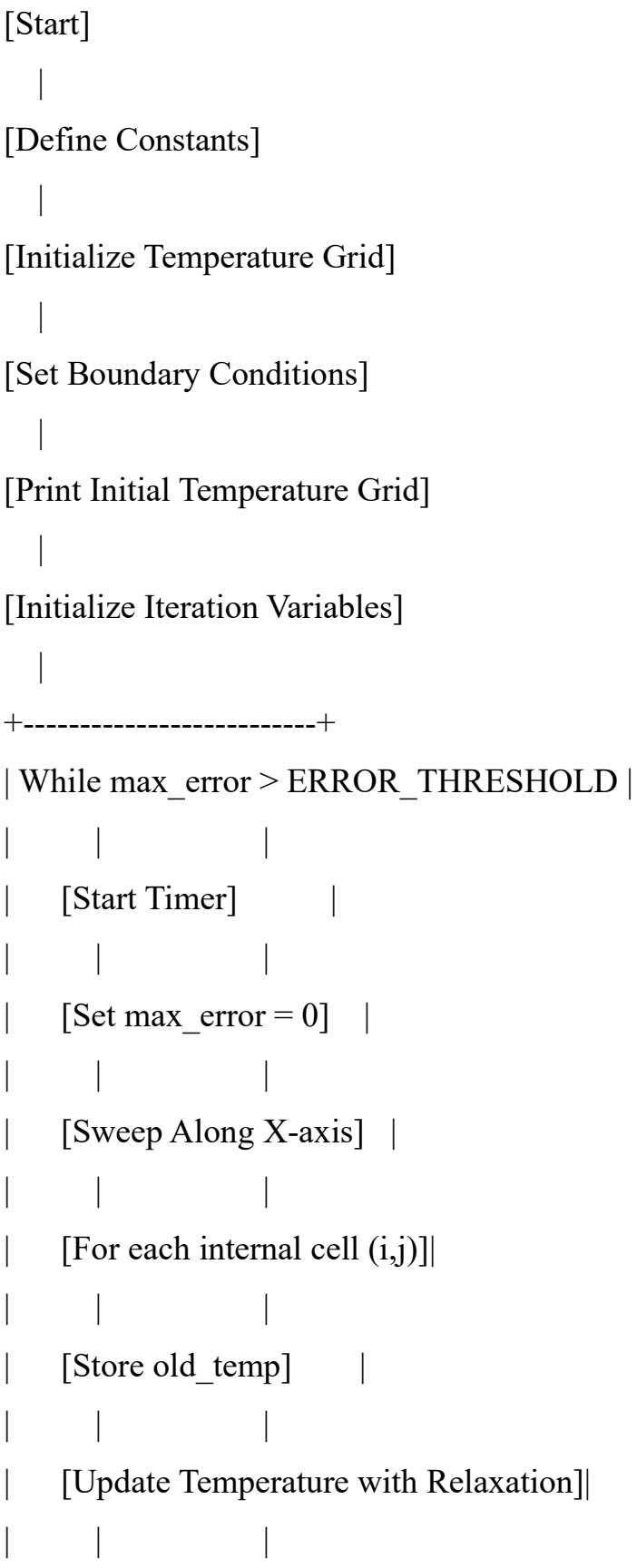
|

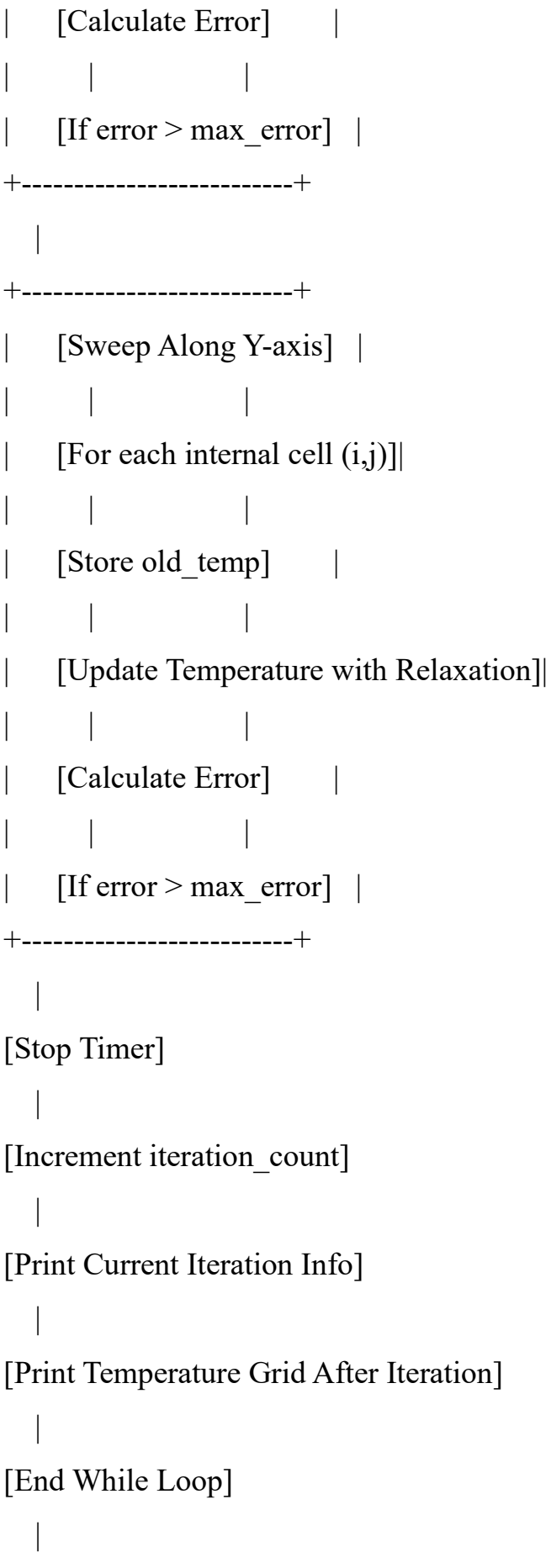
[Close Output File]

|

[End]

ADI method:


[Start]
[Define Constants]
[Initialize Temperature Grid]
[Set Boundary Conditions]
[Print Initial Temperature Grid]
[Initialize Iteration Variables]
While max_error > ERROR_THRESHOLD
[Start Timer]
[Set max_error = 0]
[Sweep Along X-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]


[Calculate Error]
[If error > max_error]
[Sweep Along Y-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Update Temperature with Relaxation]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]
[End While Loop]

[Print Final Temperature Grid]

|

[Calculate Heat Transfer]

|

[Print Total Heat Transfer]

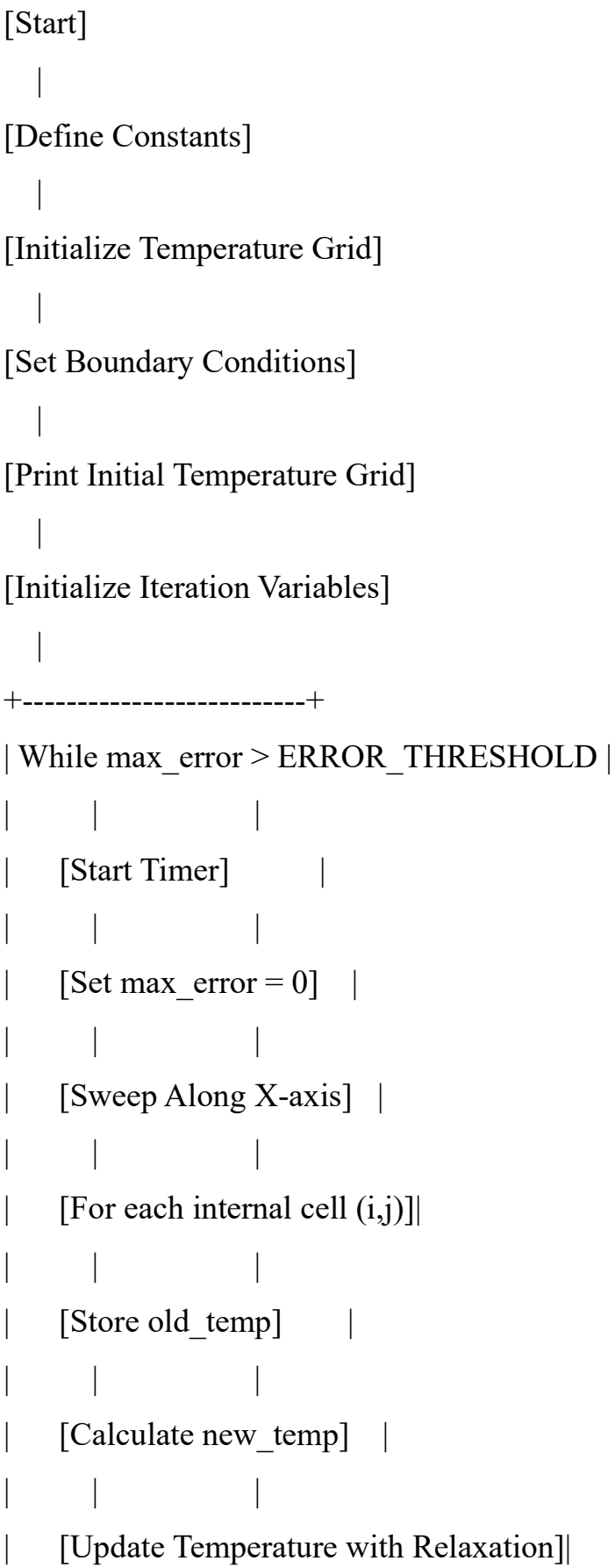
|

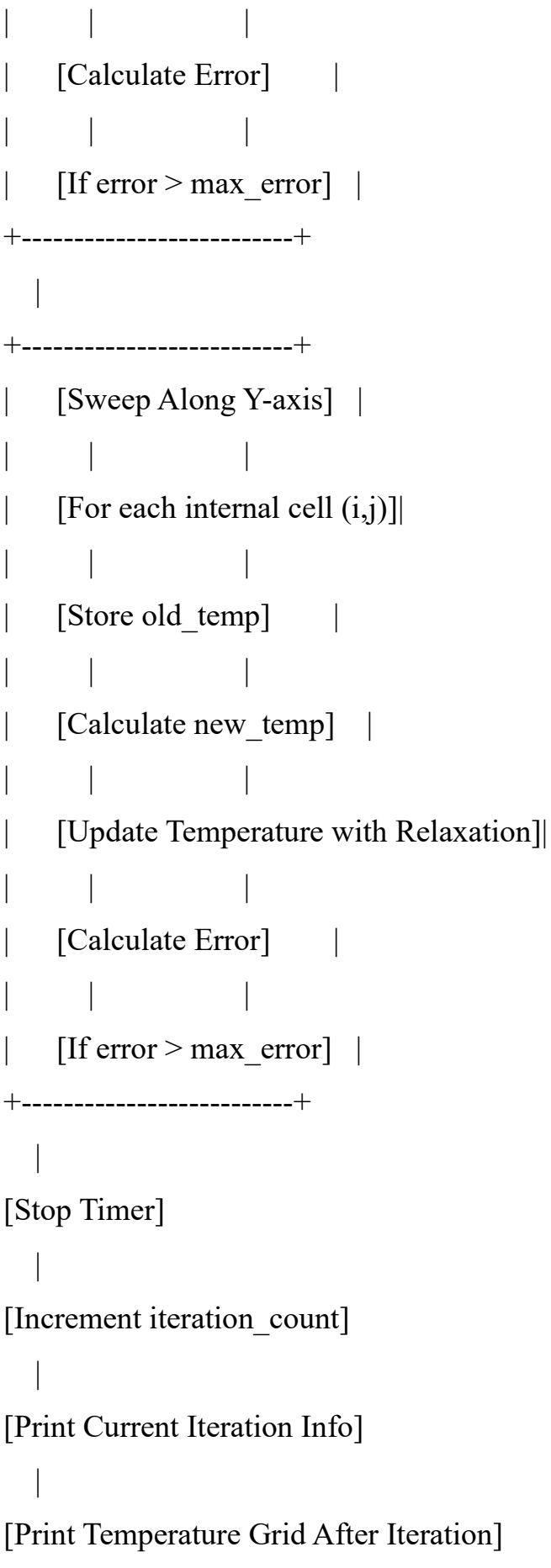
[Close Output File]

|

[End]

ADI with relaxation method:




[Calculate Error]
[If error > max_error]
[Sweep Along Y-axis]
[For each internal cell (i,j)]
[Store old_temp]
[Calculate new_temp]
[Update Temperature with Relaxation]
[Calculate Error]
[If error > max_error]
[Stop Timer]
[Increment iteration_count]
[Print Current Iteration Info]
[Print Temperature Grid After Iteration]

|

[End While Loop]

|

[Print Final Temperature Grid]

|

[Calculate Heat Transfer]

|

[Print Total Heat Transfer]

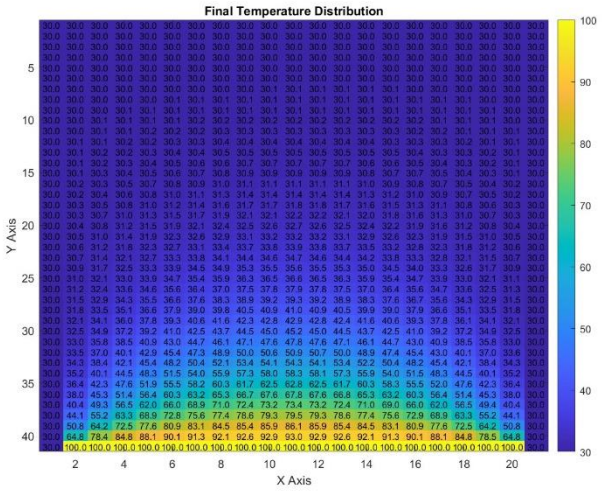
|

[Close Output File]

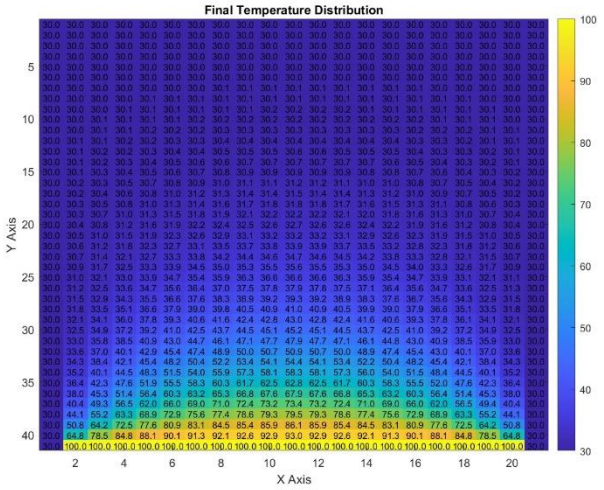
|

[End]

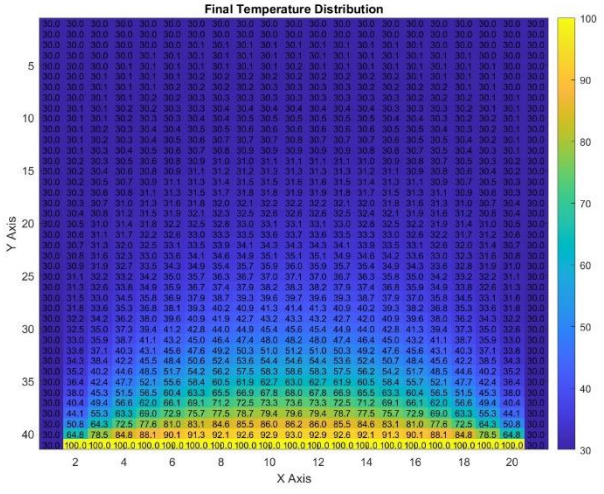
Plots:



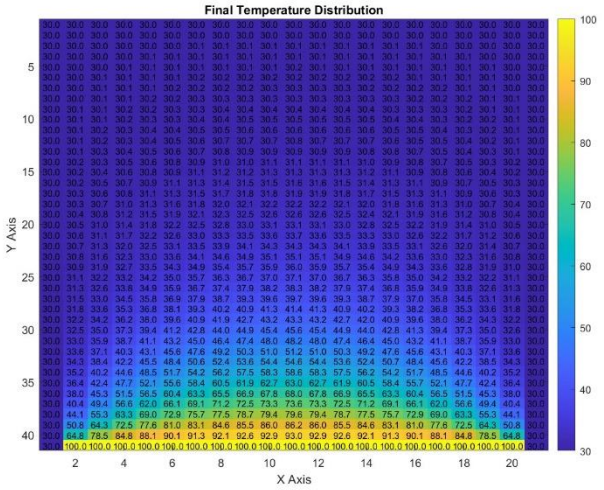
Point Gauss Siedel method



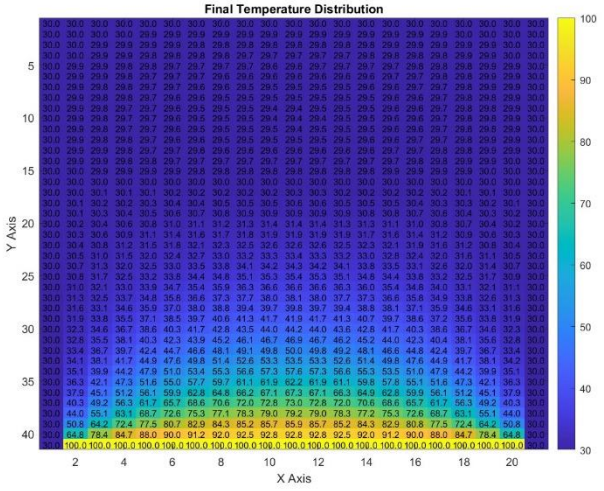
Line Gauss Siedel method



PSOR method



LSOR method



ADI method

Result:

S.no	Scheme	Grid Size	iterations	Relaxation Factor
1	Point Gauss Siedel	21 x 41	296	-
2	Line Gauss Siedel	21 x 41	149	
3	Psor	21 x 41	63	1.5
4	Lsor	21 x 41	48	1.5
5	ADI without Relaxation	21 x 41	336	
6	ADI with Relaxation	21 x 41	9	1.5

Codes:

Point Gauss Sidel Method:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41  // Number of cells in Y axis (height)
#define N 21  // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0   // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

// Temperature grid
double T[M][N];

void initializeGrid()
{
   for (int i = 0; i < M; i++)
```

```
    {
        for (int j = 0; j < N; j++)
        {
            T[i][j] = 0; // Set initial temperature to zero
        }
    }
}

// Set boundary conditions
void setBoundaryConditions()
          {
    for (int j = 0; j < N; j++)
    { // Bottom wall (X-axis)
        T[M-1][j] = T_BOTTOM; // Set bottom boundary temperature
    }
    for (int j = 0; j < N; j++)
    { // Top wall (X-axis)
        T[0][j] = T_OTHER; // Set top boundary temperature
    }
    for (int i = 0; i < M; i++)
    { // Left wall (Y-axis)
        T[i][0] = T_OTHER; // Set left boundary temperature
    }
    for (int i = 0; i < M; i++)
    { // Right wall (Y-axis)
        T[i][N-1] = T_OTHER; // Set right boundary temperature
    }
}
```

```
// Print grid to file
void printGrid(const char* title, FILE *file)
{
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

// Calculate heat transfer along the bottom boundary
double calculateHeatTransfer()
{
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++)
    {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
    }
    return total_heat_transfer * K; // Multiply by thermal conductivity
}

int main()
{
    FILE *outputFile;
```

```
outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

if (outputFile == NULL)
{
   printf("Error opening file!\n");
   return 1;
}

// Initialize the grid and set boundary conditions
initializeGrid();
setBoundaryConditions();

printGrid("Initial Temperature Grid", outputFile);

double max_error;
clock_t start_time, end_time;

int iteration_count = 0; // Counter for iterations

do {
   start_time = clock(); // Start timer
   max_error = 0.0;

   for (int i = 1; i < M - 1; i++)
   {
      for (int j = 1; j < N - 1; j++)
      {
         double old_temp = T[i][j];
         T[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]);
```

```
            // Calculate maximum error
            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
            }
        }
    }

    end_time = clock(); // End timer

    iteration_count++; // Increment iteration count

    fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
            iteration_count,
            (double)(end_time - start_time) / CLOCKS_PER_SEC);

    printGrid("Temperature Grid After Iteration", outputFile);

    printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

}
 while (max_error > ERROR_THRESHOLD); // Continue until convergence

printGrid("Final Temperature Grid", outputFile);

double heatTransfer = calculateHeatTransfer();
printf("Total Heat Transfer along Bottom Boundary: %.2f W\n",
heatTransfer);
```

```
    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n",
heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

Line Gauss Sidel method:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41 // Number of cells in Y axis (height)
#define N 21 // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0 // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

double T[M][N]; // Temperature grid

void initializeGrid() {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            T[i][j] = 0; // Set initial temperature to zero
        }
```

```
    }
}

void setBoundaryConditions() {
    for (int j = 0; j < N; j++) {
        T[M-1][j] = T_BOTTOM; // Bottom boundary temperature
    }
    for (int j = 0; j < N; j++) {
        T[0][j] = T_OTHER; // Top boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][0] = T_OTHER; // Left boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][N-1] = T_OTHER; // Right boundary temperature
    }
}

void printGrid(const char* title, FILE *file) {
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

double calculateHeatTransfer() {
```

```
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++) {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
    }
    return total_heat_transfer * K;
}

int main() {
    FILE *outputFile;
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    initializeGrid();
    setBoundaryConditions();

    printGrid("Initial Temperature Grid", outputFile);

    double max_error;
    clock_t start_time, end_time;

    int iteration_count = 0;

    do {
        start_time = clock(); // Start timer
```

```
max_error = 0.0;

// Sweep along the x-axis
for (int i = 1; i < M - 1; i++) {
    for (int j = 1; j < N - 1; j++) {
        double old_temp = T[i][j];
        T[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]);

        double error = fabs(T[i][j] - old_temp);
        if (error > max_error) {
            max_error = error;
        }
    }
}

// Sweep along the y-axis
for (int j=1; j < N - 1; j++) {
    for (int i=1; i < M - 1; i++) {
        double old_temp = T[i][j];
        T[i][j] = 0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]);

        double error = fabs(T[i][j] - old_temp);
        if (error > max_error) {
            max_error = error;
        }
    }
}

end_time = clock(); // End timer
```

```
        iteration_count++; // Increment iteration count

        fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
                iteration_count,
                (double)(end_time - start_time) / CLOCKS_PER_SEC);

        printGrid("Temperature Grid After Iteration", outputFile);

        printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

    } while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n",
heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

PSOR:

```
#include <stdio.h>
#include <math.h>
```

```
#include <time.h>

#define M 41  // Number of cells in Y axis (height)
#define N 21  // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0   // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

double T[M][N]; // Temperature grid

void initializeGrid() {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            T[i][j] = 0; // Set initial temperature to zero
        }
    }
}

void setBoundaryConditions() {
    for (int j = 0; j < N; j++) {
        T[M-1][j] = T_BOTTOM; // Bottom boundary temperature
    }
    for (int j = 0; j < N; j++) {
        T[0][j] = T_OTHER; // Top boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][0] = T_OTHER; // Left boundary temperature
    }
```

```
    for (int i = 0; i < M; i++) {
        T[i][N-1] = T_OTHER; // Right boundary temperature
    }
}

void printGrid(const char* title, FILE *file) {
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

double calculateHeatTransfer() {
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++) {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
    }
    return total_heat_transfer * K;
}

int main() {
    FILE *outputFile;
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL) {
```

```
    printf("Error opening file!\n");
    return 1;
}

initializeGrid();
setBoundaryConditions();

printGrid("Initial Temperature Grid", outputFile);

double max_error;
clock_t start_time, end_time;

int iteration_count = 0; // Counter for iterations

double omega = 1.5; // Relaxation factor, adjust as needed

do {
    start_time = clock(); // Start timer
    max_error = 0.0;

    // Sweep along the x-axis
    for (int i = 1; i < M - 1; i++) {
        for (int j = 1; j < N - 1; j++) {
            double old_temp = T[i][j];
            T[i][j] = (1 - omega) * old_temp + omega * (0.25 *(T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]));
            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
```

```
            }
        }
    }

    // Sweep along the y-axis
    for (int j=1; j < N - 1; j++) {
        for (int i=1; i < M - 1; i++) {
            double old_temp = T[i][j];
            T[i][j] = (1 - omega) * old_temp + omega * (0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]));

            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
            }
        }
    }

    end_time = clock(); // End timer

    iteration_count++; // Increment iteration count

    fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
            iteration_count,
            (double)(end_time - start_time) / CLOCKS_PER_SEC);

    printGrid("Temperature Grid After Iteration", outputFile);

    printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);
```

```
    } while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n",
heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

LSOR:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41  // Number of cells in Y axis (height)
#define N 21  // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0   // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

// Temperature grid
```

```
double T[M][N];


void initializeGrid()
{
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            T[i][j] = 0; // Set initial temperature to zero
        }
    }
}


// Set boundary conditions
void setBoundaryConditions()
{
    for (int j = 0; j < N; j++)
    {
        T[M-1][j] = T_BOTTOM; // Set bottom boundary temperature starting from
the first cell
    }
    for (int j = 0; j < N; j++)
    {
        T[0][j] = T_OTHER; // Set top boundary temperature
    }
    for (int i = 0; i < M; i++)
    {
        T[i][0] = T_OTHER; // Set left boundary temperature
    }
```

```
    for (int i = 0; i < M; i++)
    {
        T[i][N-1] = T_OTHER; // Set right boundary temperature
    }
}

// Print grid to file
void printGrid(const char* title, FILE *file)
{
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

// Calculate heat transfer along the bottom boundary
double calculateHeatTransfer()
{
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++)
    {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
    }
```

```
    return total_heat_transfer * K; // Multiply by thermal conductivity
}

int main()
{
    FILE *outputFile;
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }

    initializeGrid();
    setBoundaryConditions();

    printGrid("Initial Temperature Grid", outputFile);

    double max_error;
    clock_t start_time, end_time;

    int iteration_count = 0; // Counter for iterations
    double omega = 1.5; // Relaxation factor

    do {
        start_time = clock(); // Start timer
        max_error = 0.0;
```

```
        // Sweep along the x-axis
        for (int i = 1; i < M - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                double old_temp = T[i][j];
                T[i][j] = (1 - omega) * old_temp + omega * (0.25 *(T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]));
                double error = fabs(T[i][j] - old_temp);
                if (error > max_error) {
                    max_error = error;
                }
            }
        }

        // Sweep along the y-axis
        for (int j=1; j < N - 1; j++) {
            for (int i=1; i < M - 1; i++) {
                double old_temp = T[i][j];
                T[i][j] = (1 - omega) * old_temp + omega * (0.25 * (T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]));

                double error = fabs(T[i][j] - old_temp);
                if (error > max_error) {
                    max_error = error;
                }
            }
        }

        end_time = clock(); // End timer
```

```
        iteration_count++; // Increment iteration count

        fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
                iteration_count,
                (double)(end_time - start_time) / CLOCKS_PER_SEC);

        printGrid("Temperature Grid After Iteration", outputFile);

        printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

    } while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n",
heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

ADI Without Relaxation:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41  // Number of cells in Y axis (height)
#define N 21  // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0   // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

double T[M][N]; // Temperature grid

void initializeGrid() {
   for (int i = 0; i < M; i++) {
      for (int j = 0; j < N; j++) {
         T[i][j] = 0; // Set initial temperature to zero
      }
   }
}

void setBoundaryConditions() {
   for (int j = 0; j < N; j++) {
      T[M-1][j] = T_BOTTOM; // Set bottom boundary temperature
   }
   for (int j = 0; j < N; j++) {
      T[0][j] = T_OTHER; // Set top boundary temperature
   }
```

```
    for (int i = 0; i < M; i++) {
        T[i][0] = T_OTHER; // Set left boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][N-1] = T_OTHER; // Set right boundary temperature
    }
}

void printGrid(const char* title, FILE *file) {
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

double calculateHeatTransfer() {
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++) {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
    }
    return total_heat_transfer * K; // Multiply by thermal conductivity
}

int main() {
    FILE *outputFile;
```

```
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    initializeGrid();
    setBoundaryConditions();

    printGrid("Initial Temperature Grid", outputFile);

    double max_error;
    clock_t start_time, end_time;

    int iteration_count = 0;

    double iteration_level = 0.5; // Fixed iteration level

    do {
        start_time = clock(); // Start timer
        max_error = 0.0;

         // Sweep along the x-axis
        for (int i = 1; i < M - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                double old_temp = T[i][j];
                T[i][j] = ((T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]) / 4) *
iteration_level + old_temp * (1 - iteration_level);
```

```
            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
            }
        }
    }

    // Sweep along the y-axis
    for (int j=1; j < N - 1; j++) {
        for (int i=1; i < M - 1; i++) {
            double old_temp = T[i][j];
            T[i][j] = ((T[i + 1][j] + T[i - 1][j] + T[i][j + 1] + T[i][j - 1]) / 4) *
iteration_level + old_temp * (1 - iteration_level);

            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
            }
        }
    }

    end_time = clock(); // End timer

    iteration_count++; // Increment iteration count

    fprintf(outputFile, "Iteration %d Max error: %.6f\n",
            iteration_count, max_error);
```

```
        printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

        // Print the grid after each iteration
        char title[50];
        snprintf(title, sizeof(title), "Temperature Grid After Iteration %d", iteration_count);
        printGrid(title, outputFile);

    } while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

ADI with Relaxation:

```
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41  // Number of cells in Y axis (height)
#define N 21  // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0   // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

double T[M][N]; // Temperature grid

void initializeGrid() {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            T[i][j] = 0; // Set initial temperature to zero
        }
    }
}

void setBoundaryConditions() {
    for (int j = 0; j < N; j++) {
        T[M-1][j] = T_BOTTOM; // Set bottom boundary temperature
    }
```

```
    for (int j = 0; j < N; j++) {
        T[0][j] = T_OTHER; // Set top boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][0] = T_OTHER; // Set left boundary temperature
    }
    for (int i = 0; i < M; i++) {
        T[i][N-1] = T_OTHER; // Set right boundary temperature
    }
}

void printGrid(const char* title, FILE *file) {
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

double calculateHeatTransfer() {
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
    for (int j = 0; j < N - 1; j++) {
        total_heat_transfer += (T[M-1][j] + T[M-1][j+1]) / 2 * dx;
```

```
    }
    return total_heat_transfer * K; // Multiply by thermal conductivity
}

int main() {
    FILE *outputFile;
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    initializeGrid();
    setBoundaryConditions();

    printGrid("Initial Temperature Grid", outputFile);

    double max_error;
    clock_t start_time, end_time;

    int iteration_count = 0;
        double iteration_level = 0.5; // Fixed iteration level set to 0.5
    double omega = 1.5; // Relaxation factor

    do {
```

```
        start_time = clock(); // Start timer
        max_error = 0.0;

         // Sweep along the x-axis
        for (int i = 1; i < M - 1; i++) {
            for (int j = 1; j < N - 1; j++) {
                double old_temp = T[i][j];
                double new_temp = ((T[i + 1][j] + T[i - 1][j] + T[i][j + 1] +
T[i][j - 1]) / 4);

                // Apply relaxation factor and iteration level
                T[i][j] = (1 - omega) * old_temp + omega * new_temp *
iteration_level;

                double error = fabs(T[i][j] - old_temp);
                if (error > max_error) {
                    max_error = error;
                }
            }
        }

        // Sweep along the y-axis
        for (int j=1; j < N - 1; j++) {
            for (int i=1; i < M - 1; i++) {
                double old_temp = T[i][j];
                double new_temp = ((T[i + 1][j] + T[i - 1][j] + T[i][j + 1] +
T[i][j - 1]) / 4);
```

```
            // Apply relaxation factor and iteration level
            T[i][j] = (1 - omega) * old_temp + omega * new_temp * iteration_level;

            double error = fabs(T[i][j] - old_temp);
            if (error > max_error) {
                max_error = error;
            }
        }
    }

    end_time = clock(); // End timer

    iteration_count++; // Increment iteration count

    fprintf(outputFile, "Iteration %d Max error: %.6f\n",
            iteration_count, max_error);

    printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

    // Print the grid after each iteration
    char title[50];
    snprintf(title, sizeof(title), "Temperature Grid After Iteration %d", iteration_count);
    printGrid(title, outputFile);
```

```
    } while (max_error > ERROR_THRESHOLD); // Continue until convergence


    printGrid("Final Temperature Grid", outputFile);


    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);


    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);


    fclose(outputFile); // Close the file


    return 0;
}
```